

Авторизация и Аутентификация

Аутентификация и авторизация — в чём разница

Это два разных, хотя и связанных понятия:

- **Аутентификация** — проверка того, что пользователь действительно тот, за кого себя выдаёт (например, вход по логину и паролю).
- **Авторизация** — определение того, к каким разделам и функциям приложения у пользователя есть доступ.

Пример: обычный пользователь и администратор — оба прошли аутентификацию (вошли в систему), но авторизация у них разная: администратор может видеть и изменять данные всех пользователей, а не только свои.

Сначала всегда выполняется **аутентификация**, и только потом — **авторизация**.

Для реализации авторизации и аутентификации часто используют **JWT (JSON Web Token)** — токен, который позволяет безопасно передавать данные между клиентом и сервером в зашифрованном виде. Сначала разберём базовый вариант с одним токеном доступа, затем — пару токенов (токен доступа и токен обновления) для большей безопасности, а в конце добавим авторизацию с ограничением доступа для разных категорий пользователей.

Механизм сессий (через cookie)

HTTP (HyperText Transfer Protocol — протокол передачи гипертекста) не сохраняет состояние между запросами: сервер не знает, что предыдущий запрос был от того же пользователя. Из-за этого, без дополнительных механизмов, невозможно, например, остаться залогиненным при переходе между страницами сайта.

Для решения этой проблемы используют **cookie**-файлы и сессии. Cookie — небольшой фрагмент данных (до 4 КБ), который сервер отправляет браузеру, а тот сохраняет его на стороне пользователя на определённый срок. Чаще всего в cookie хранится просто уникальный идентификатор пользователя.

Как работает авторизация через сессии и cookie

1. Пользователь вводит логин и пароль и отправляет их на сервер.
2. Сервер проверяет данные и, если всё верно, создаёт уникальный идентификатор сессии и сохраняет его в своей базе данных.
3. Сервер отправляет этот идентификатор клиенту в виде cookie.
4. При каждом следующем запросе браузер автоматически отправляет этот cookie на сервер.
5. Сервер по идентификатору сессии определяет, какой это пользователь, и аутентифицирует его.
6. При выходе из системы (или истечении срока) идентификатор удаляется и на сервере, и в cookie у клиента.

Такой подход к аутентификации, например, использует фреймворк Django.

Недостатки механизма сессий

НЕДОСТАТОК	В ЧЁМ ПРОБЛЕМА
Хранение состояния	Сессии нужно хранить на сервере — это усложняет масштабирование, если серверов несколько.
Уязвимость к XSS (Cross-Site Scripting — межсайтовый скриптинг)	Если сайт уязвим к XSS-атаке, злоумышленник может украсть идентификатор сессии и получить доступ к аккаунту.
Уязвимость к CSRF (Cross-Site Request Forgery — подделка межсайтового запроса)	Без защиты злоумышленник может выполнять действия от имени пользователя без его ведома.
Ограниченная передача данных	Идентификатор сессии передаётся только через cookie — неудобно при работе с разных устройств и платформ.
Ограниченный срок жизни	Данные сессии могут удаляться или "протухать" по настройкам сервера — пользователь неожиданно теряет доступ.

Cookie также можно подделать или скопировать, что тоже риск. Снизить его помогают безопасные методы хранения и передачи: шифрование, а также флаги **HttpOnly** (запрещает доступ к cookie через JavaScript) и **Secure** (cookie передаётся только по HTTPS).

JSON Web Token (JWT)

На смену механизму сессий пришёл **JWT (JSON Web Token)** — стандартный формат токена для аутентификации и авторизации, который хранит информацию о пользователе в формате JSON и подписывает её.

Принцип работы: сервер создаёт JWT с помощью секретного ключа и отправляет его клиенту. Клиент сохраняет токен (обычно в локальном хранилище браузера) и добавляет его в заголовок каждого запроса. Сервер проверяет токен при каждом запросе и отвечает клиенту.

Главное отличие от сессий: состояние пользователя не хранится на сервере — оно хранится внутри самого токена, на стороне клиента.

ПРЕИМУЩЕСТВО	ЧТО ЭТО ДАЁТ
Без сохранения состояния	Серверу не нужно хранить сессии — легче распределять нагрузку между серверами и масштабировать приложение.
Гибкая передача данных	Токен можно передавать не только через cookie, а и в теле запроса или в заголовке.
Безопасность	Токен имеет подпись — можно проверить, что его не подделали. Дополнительно его можно зашифровать.
Удобство	Стандартизированный формат — легко интегрируется в разные системы и платформы.

У JWT есть и недостатки: токен нельзя досрочно отозвать, а изменить информацию внутри него можно только полной заменой на новый токен. Поэтому JWT обычно используют вместе с другими методами защиты.

Структура токена

JWT — это строка вида `header.payload.signature`, три части, разделённые точкой. Формат описан в открытом стандарте RFC 7519 (Request for Comments 7519 — документ, описывающий стандарт).

Пример токена:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG91IiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxw...
```

- header** — алгоритм хеширования, который использовался для подписи:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **payload** — данные, которые нужно передать (например, id пользователя):

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

- **signature** — подпись из header и payload, зашифрованных секретным ключом по алгоритму из header.

👉 **payload не защищён** — это просто кодировка Base64, его можно раскодировать без секретного ключа. Поэтому в JWT нельзя передавать пароли и другую конфиденциальную информацию.

Библиотека python-jose

python-jose — библиотека для создания, подписи, проверки и расшифровки JWT в Python.

```
poetry add python-jose[cryptography]
```

```
from jose import jwt

# данные для токена
payload = {"sub": "1234567890", "name": "John Doe"}

# создание токена с симметричным ключом
encoded = jwt.encode(payload, "secret_key", algorithm='HS256')
print(encoded)

# проверка токена
decoded = jwt.decode(encoded, "secret_key", algorithms=['HS256'])
print(decoded)
```

Если при decode указать неверный секретный ключ — библиотека выбросит ошибку `JWTError: "Signature verification failed"`.

Базовый алгоритм JWT-авторизации

1. Клиент отправляет на сервер логин (`username`) и пароль (`password`).
2. Сервер проверяет данные и, если всё верно, создаёт JWT-токен и отправляет его клиенту.
3. Клиент сохраняет токен и дальше передаёт его в заголовке каждого запроса для аутентификации.
4. Сервер проверяет подпись и срок действия токена. Если токен верный — обрабатывает запрос; если нет — возвращает ошибку.

Реализация на FastAPI

Нужные пакеты: **FastAPI** — фреймворк; **python-jose** — создание и проверка JWT; **passlib** — хеширование и проверка паролей (с алгоритмом bcrypt); **sqlalchemy** — ORM (Object-Relational Mapping — библиотека для работы с базой данных как с объектами Python).

```
poetry add fastapi "python-jose[cryptography]" "passlib[bcrypt]" sqlalchemy
```

Код разбит на три файла: `db.py` — подключение к базе и модели, `auth.py` — аутентификация и авторизация, `app.py` — маршруты приложения.

`db.py` — подключение к базе данных

```
from sqlalchemy import create_engine, Column, String, Integer
from sqlalchemy.orm import sessionmaker, DeclarativeBase, mapped_column, Mapped

DATABASE_URL = "sqlite:///./sql_app.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    username: Mapped[str] = mapped_column(String(150), nullable=False, unique=True)
    password: Mapped[str] = mapped_column(String(255), nullable=False)

Base.metadata.create_all(bind=engine)

# dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Создаётся SQLite-база в файле `sql_app.db`. Параметр `check_same_thread: False` нужен потому, что SQLite по умолчанию разрешает соединение из одного потока — этот параметр снимает ограничение.

`auth.py` — аутентификация и токены

```

from datetime import datetime, timedelta, UTC
from typing import Optional

from fastapi import Depends, HTTPException, status
from passlib.context import CryptContext
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
from jose import JWTError, jwt

from db import get_db, User

SECRET_KEY = "secret_key"
ALGORITHM = "HS256"

class Hash:
    pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

    def verify_password(self, plain_password, hashed_password):
        return self.pwd_context.verify(plain_password, hashed_password)

    def get_password_hash(self, password: str):
        return self.pwd_context.hash(password)

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/login")

# создание нового access-токена
async def create_access_token(data: dict, expires_delta: Optional[float] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(UTC) + timedelta(seconds=expires_delta)
    else:
        expire = datetime.now(UTC) + timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(
    token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)
):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:
        # расшифровка JWT
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload["sub"]
        if username is None:
            raise credentials_exception
    except JWTError as e:
        raise credentials_exception

    user: User = db.query(User).filter(User.username == username).first()
    if user is None:
        raise credentials_exception
    return user

```

Коротко о ключевых частях:

- `OAuth2PasswordBearer(tokenUrl="/login")` — встроенный в FastAPI механизм для OAuth2 (Open Authorization 2.0 — открытый стандарт авторизации). Он сам достаёт токен из заголовка `Authorization` запроса и передаёт его в функцию. Если токен отсутствует или недействителен — сразу возвращает ошибку 401.
- `Hash` — хеширует и проверяет пароли через `bcrypt`. Хеш нельзя обратно превратить в пароль, поэтому даже при утечке базы данных сами пароли остаются в безопасности.
- `create_access_token` — создаёт токен со сроком жизни: 15 минут по умолчанию, либо указанный вручную.
- `get_current_user` — расшифровывает токен, достаёт из него имя пользователя (`sub`) и ищет пользователя в базе. Если токен не прошёл проверку или пользователь не найден — возвращает ошибку 401.

`app.py` — маршруты приложения

```

from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session
from pydantic import BaseModel

from auth import create_access_token, get_current_user, Hash
from db import User, get_db

app = FastAPI()
hash_handler = Hash()

class UserModel(BaseModel):
    username: str
    password: str

@app.post("/signup")
async def signup(body: UserModel, db: Session = Depends(get_db)):
    exist_user = db.query(User).filter(User.username == body.username).first()
    if exist_user:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT, detail="Account already exists"
        )
    new_user = User(
        username=body.username, password=hash_handler.get_password_hash(body.password)
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return {"new_user": new_user.username}

@app.post("/login")
async def login(
    body: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)
):
    user = db.query(User).filter(User.username == body.username).first()
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid username"
        )
    if not hash_handler.verify_password(body.password, user.password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid password"
        )
    # генерация JWT
    access_token = await create_access_token(data={"sub": user.username})
    return {"access_token": access_token, "token_type": "bearer"}

@app.get("/")
async def root():
    return {"message": "Hello World"}

@app.get("/secret")
async def read_item(current_user: User = Depends(get_current_user)):
    return {"message": "secret router", "owner": current_user.username}

```

Маршруты приложения:

МАРШРУТ	НАЗНАЧЕНИЕ
POST /signup	Регистрация: сохраняет нового пользователя с захешированным паролем.
POST /login	Проверяет логин/пароль и возвращает <code>access_token</code> .
GET /	Открытый маршрут, доступен без авторизации.
GET /secret	Защищённый маршрут — доступен только с валидным <code>access_token</code> .

❗ Данные для входа передаются как `username` и `password` — этого требует стандарт OAuth2. Даже если в базе данных поле называется `email`, для маршрута логина всё равно нужно использовать имена `username` / `password`, иначе Swagger-документация и `OAuth2PasswordBearer` не смогут корректно забрать токен из запроса.

Проверить работу можно через встроенную Swagger-документацию FastAPI: сначала регистрируем пользователя через `/signup`, затем авторизуемся через `/login` (или кнопку «Authorize»), получаем `access_token`, и после этого маршрут `/secret` становится доступен. Без токена этот маршрут вернёт ошибку 401.

Пара JWT-токенов: access + refresh

Если у токена долгий срок жизни, то украв его, злоумышленник сможет пользоваться доступом всё это время. Чтобы снизить риск, используют не один токен, а пару:

- **access token** — используется для авторизации запросов, живёт недолго (например, 15 минут).
- **refresh token** — выдаётся один раз при входе и используется только для того, чтобы получить новый `access token`, когда старый истёк. Живёт значительно дольше (например, 7 дней).

Так пользователю не нужно каждый раз вводить логин и пароль заново — а срок жизни `access token` достаточно короткий, чтобы кража токена не была большой проблемой.

Схема работы

1. Клиент отправляет логин и пароль на сервер.
2. Сервер проверяет данные и, если всё верно, генерирует пару токенов (`access + refresh`) и отправляет их клиенту.
3. Клиент использует `access token` для запросов — так же, как и в базовом варианте.
4. Когда срок жизни `access token` истекает, клиент отправляет `refresh token` на отдельный маршрут обновления.
5. Сервер проверяет `refresh token` и, если он валиден, выдаёт новый `access token`.

db.py — изменения

В модель `User` добавляется поле для хранения `refresh`-токена:

```
refresh_token: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
```

`Refresh`-токен хранится в базе данных, чтобы сервер мог проверить его подлинность и при необходимости отозвать (например, если он скомпрометирован). В этой реализации на пользователя хранится один `refresh`-токен — то есть только одно активное устройство. Для нескольких устройств понадобится отдельная таблица токенов на каждое устройство.

auth.py — создание и проверка токенов


```

from datetime import datetime, timedelta, UTC
from typing import Optional, Literal

from fastapi import Depends, HTTPException, status
from passlib.context import CryptContext
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
from jose import JWTError, jwt

from db import get_db, User

SECRET_KEY = "secret_key"
ALGORITHM = "HS256"
REFRESH_TOKEN_EXPIRE_MINUTES = 60 * 24 * 7 # 7 дней
ACCESS_TOKEN_EXPIRE_MINUTES = 15

class Hash:
    pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

    def verify_password(self, plain_password, hashed_password):
        return self.pwd_context.verify(plain_password, hashed_password)

    def get_password_hash(self, password: str):
        return self.pwd_context.hash(password)

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/login")

def create_token(
    data: dict, expires_delta: timedelta, token_type: Literal["access", "refresh"]
):
    to_encode = data.copy()
    now = datetime.now(UTC)
    expire = now + expires_delta
    to_encode.update({"exp": expire, "iat": now, "token_type": token_type})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

async def create_access_token(data: dict, expires_delta: Optional[float] = None):
    if expires_delta:
        access_token = create_token(data, expires_delta, "access")
    else:
        access_token = create_token(
            data, timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES), "access"
        )
    return access_token

async def create_refresh_token(data: dict, expires_delta: Optional[float] = None):
    if expires_delta:
        refresh_token = create_token(data, expires_delta, "refresh")
    else:
        refresh_token = create_token(
            data, timedelta(minutes=REFRESH_TOKEN_EXPIRE_MINUTES), "refresh"
        )
    return refresh_token

async def get_current_user(
    token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)
):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )

```

```

try:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    username: str = payload.get("sub")
    token_type: str = payload.get("token_type")
    if username is None or token_type != "access":
        raise credentials_exception
except JWTError as e:
    raise credentials_exception

user: User = db.query(User).filter(User.username == username).first()
if user is None:
    raise credentials_exception
return user

def verify_refresh_token(refresh_token: str, db: Session):
    try:
        payload = jwt.decode(refresh_token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        token_type: str = payload.get("token_type")
        if username is None or token_type != "refresh":
            return None
        user = (
            db.query(User)
            .filter(User.username == username, User.refresh_token == refresh_token)
            .first()
        )
        return user
    except JWTError:
        return None

```

Что изменилось по сравнению с базовым вариантом:

- Общая функция `create_token` добавляет в токен поля `iat` (issued at — время выдачи токена), `exp` (expire — время истечения) и `token_type` ("access" или "refresh").
- `get_current_user` теперь дополнительно проверяет, что тип токена именно "access" — это не даёт использовать refresh-токен вместо access-токена.
- `verify_refresh_token` проверяет, что тип токена "refresh" , и что он совпадает с тем, что сохранён в базе данных для этого пользователя — так можно отозвать конкретный refresh-токен.

Поле `iat` полезно не только для информации: оно даёт возможность вести аудит (когда был выдан токен), добавлять дополнительные проверки безопасности (например, отклонять токены, выданные до определённого момента) и реализовывать функции вроде «выйти со всех устройств, кроме текущего».

app.py — маршруты

```

from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session
from pydantic import BaseModel

from auth import (
    create_access_token,
    create_refresh_token,
    get_current_user,
    Hash,
    verify_refresh_token,
)
from db import User, get_db

app = FastAPI()
hash_handler = Hash()

class UserModel(BaseModel):
    username: str
    password: str

class TokenModel(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str

class TokenRefreshRequest(BaseModel):
    refresh_token: str

@app.post("/login", response_model=TokenModel)
async def login(
    body: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)
):
    user = db.query(User).filter(User.username == body.username).first()
    if user is None or not hash_handler.verify_password(body.password, user.password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid credentials"
        )

    access_token = await create_access_token(data={"sub": user.username})
    refresh_token = await create_refresh_token(data={"sub": user.username})
    user.refresh_token = refresh_token
    db.commit()
    return {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer",
    }

@app.post("/refresh-token", response_model=TokenModel)
async def new_token(request: TokenRefreshRequest, db: Session = Depends(get_db)):
    user = verify_refresh_token(request.refresh_token, db)
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired refresh token",
        )
    new_access_token = await create_access_token(data={"sub": user.username})
    return {
        "access_token": new_access_token,
        "refresh_token": request.refresh_token,
        "token_type": "bearer",
    }

```

```
@app.get("/secret")
async def read_item(current_user: User = Depends(get_current_user)):
    return {"message": "secret router", "owner": current_user.username}
```

`/signup` , `/` остаются без изменений. В `/login` теперь создаются и возвращаются оба токена, а `refresh_token` сохраняется в базу данных. Новый маршрут `/refresh-token` принимает refresh-токен, проверяет его через `verify_refresh_token` и, если он валиден, выдаёт новый `access_token` — без повторного ввода логина и пароля.

Обновлять ли refresh-токен вместе с access-токеном?

ПОДХОД	КАК РАБОТАЕТ	КОГДА УДОБЕН
Refresh-токен не меняется	При обновлении выдаётся только новый <code>access_token</code> , старый <code>refresh_token</code> остаётся прежним.	Пользователь обязательно переавторизуется каждые 7 дней — полезно, когда важна регулярная проверка подлинности.
Refresh-токен обновляется тоже	При каждом обновлении выдаются новые <code>access_token</code> и <code>refresh_token</code> .	Активный пользователь может оставаться авторизованным неограниченно долго — удобнее для UX (User Experience — опыт использования).

Проверка через Postman

Swagger-документация FastAPI не умеет удобно работать с refresh-токенами, поэтому для проверки такой схемы обычно используют **Postman** — отдельное приложение для отправки запросов к API и просмотра ответов сервера.

Порядок проверки: регистрация (`POST /signup`) → вход (`POST /login` , тело в формате `application/x-www-form-urlencoded`) — в ответ приходит пара токенов → запрос к `GET /secret` с заголовком `Authorization: Bearer {access_token}` → после истечения access-токена запрос к `POST /refresh-token` с заголовком `Authorization: Bearer {refresh_token}` , который возвращает новый `access_token` .

Добавляем авторизацию в реальное приложение

Теперь применим всё это к настоящему REST API — приложению заметок (Note) и тегов (Tag). Цель: у каждого пользователя должны быть только его собственные заметки и теги, недоступные другим.

Модель User и связь с Note / Tag

Файл `/src/database/models.py` :

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String, unique=True)
    email = Column(String, unique=True)
    hashed_password = Column(String)
    created_at = Column(DateTime, default=func.now())
    avatar = Column(String(255), nullable=True)
```

В модель `Note` добавляется внешний ключ на пользователя и связь:

```
class Note(Base):
    # ...
    user_id = Column("user_id", ForeignKey("users.id", ondelete="CASCADE"), default=None)
    user = relationship("User", backref="notes")
```

`backref="notes"` позволяет получить все заметки пользователя как `user.notes` .

Такое же поле добавляется и в `Tag` — но здесь важно ещё одно изменение:

```
class Tag(Base):
    __tablename__ = "tags"
    __table_args__ = (UniqueConstraint("name", "user_id", name="unique_tag_user"),)
    # ...
    user_id = Column("user_id", ForeignKey("users.id", ondelete="CASCADE"), default=None)
    user = relationship("User", backref="tags")
```

Раньше поле `name` у тега было уникальным глобально — два разных пользователя не могли создать тег с одинаковым названием. Теперь уникальность проверяется по паре `(name, user_id)` — название тега уникально только в пределах одного пользователя.

Миграция базы данных (Alembic)

После изменения моделей нужно создать и применить миграцию:

```
alembic revision --autogenerate -m "add user"
alembic upgrade head
```

Первая команда сравнивает модели с текущей структурой базы данных и автоматически генерирует файл миграции (новая таблица `users`, новые колонки `user_id`, новые внешние ключи и уникальное ограничение). Вторая — применяет все неприменённые миграции к базе данных.

Схемы валидации

Файл `src/schemas.py`:

```
class User(BaseModel):
    id: int
    username: str
    email: str
    avatar: str

    model_config = ConfigDict(from_attributes=True)

class UserCreate(BaseModel):
    username: str
    email: str
    password: str

class Token(BaseModel):
    access_token: str
    token_type: str
```

- `User` — то, что отдаём клиенту: без хеша пароля, только безопасные поля. `ConfigDict(from_attributes=True)` позволяет Pydantic строить схему прямо из объекта SQLAlchemy.
- `UserCreate` — то, что принимаем при регистрации.
- `Token` — формат ответа с токеном доступа.

Репозиторий пользователя

Файл `src/repository/users.py` — слой доступа к данным, отделённый от бизнес-логики:

```

class UserRepository:
    def __init__(self, session: AsyncSession):
        self.db = session

    async def get_user_by_id(self, user_id: int) -> User | None:
        stmt = select(User).filter_by(id=user_id)
        user = await self.db.execute(stmt)
        return user.scalar_one_or_none()

    async def get_user_by_username(self, username: str) -> User | None:
        stmt = select(User).filter_by(username=username)
        user = await self.db.execute(stmt)
        return user.scalar_one_or_none()

    async def get_user_by_email(self, email: str) -> User | None:
        stmt = select(User).filter_by(email=email)
        user = await self.db.execute(stmt)
        return user.scalar_one_or_none()

    async def create_user(self, body: UserCreate, avatar: str = None) -> User:
        user = User(
            **body.model_dump(exclude_unset=True, exclude={"password"}),
            hashed_password=body.password,
            avatar=avatar
        )
        self.db.add(user)
        await self.db.commit()
        await self.db.refresh(user)
        return user

```

`create_user` распаковывает все поля из `UserCreate`, кроме пароля (`exclude={"password"}`) — пароль передаётся отдельно, уже как хеш, в поле `hashed_password`.

Конфигурация JWT

Файл `/src/conf/config.py` :

```

class Config:
    DB_URL = "postgresql+asyncpg://postgres:567234@localhost:5432/todo_app"
    JWT_SECRET = "your_secret_key"
    JWT_ALGORITHM = "HS256"
    JWT_EXPIRATION_SECONDS = 3600 # 1 час

config = Config

```

`JWT_SECRET` в реальном проекте должен быть сложным и храниться в безопасном месте (не в коде) — это будет исправлено в следующем разделе.

Сервис пользователя (Gravatar-аватар)

Файл `/src/services/users.py` :

```

class UserService:
    def __init__(self, db: AsyncSession):
        self.repository = UserRepository(db)

    async def create_user(self, body: UserCreate):
        avatar = None
        try:
            g = Gravatar(body.email)
            avatar = g.get_image()
        except Exception as e:
            print(e)

        return await self.repository.create_user(body, avatar)

    async def get_user_by_username(self, username: str):
        return await self.repository.get_user_by_username(username)

    async def get_user_by_email(self, email: str):
        return await self.repository.get_user_by_email(email)

```

Gravatar (Globally Recognized Avatar — глобально распознаваемый аватар) — бесплатный сервис, который привязывает картинку-аватар к email. Пакет `libgravatar` по email пользователя получает URL готового аватара — не нужно самим хранить и загружать изображения.

Сервис аутентификации

Файл `/src/services/auth.py` — хеширование паролей, создание и проверка JWT:

```

class Hash:
    pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

    def verify_password(self, plain_password, hashed_password):
        return self.pwd_context.verify(plain_password, hashed_password)

    def get_password_hash(self, password: str):
        return self.pwd_context.hash(password)

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")

async def create_access_token(data: dict, expires_delta: Optional[int] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(UTC) + timedelta(seconds=expires_delta)
    else:
        expire = datetime.now(UTC) + timedelta(seconds=config.JWT_EXPIRATION_SECONDS)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(
        to_encode, config.JWT_SECRET, algorithm=config.JWT_ALGORITHM
    )
    return encoded_jwt

async def get_current_user(
    token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)
):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(
            token, config.JWT_SECRET, algorithms=[config.JWT_ALGORITHM]
        )
        username = payload["sub"]
        if username is None:
            raise credentials_exception
    except JWTError as e:
        raise credentials_exception
    user_service = UserService(db)
    user = await user_service.get_user_by_username(username)
    if user is None:
        raise credentials_exception
    return user

```

`tokenUrl="/api/auth/login"` в `oauth2_scheme` нужен, чтобы Swagger-документация знала, через какой маршрут получать токен для кнопки «Authorize».

Заметьте: этот модуль — просто набор функций, а не класс, в отличие от остальных сервисов проекта. Так сделано осознанно: у аутентификации нет состояния между вызовами, функции проще тестировать отдельно, и это делает код читаемее для операций, связанных с токенами и безопасностью.

Маршруты аутентификации

Файл `src/api/auth.py` :


```

router = APIRouter(prefix="/auth", tags=["auth"])

@router.post("/register", response_model=User, status_code=status.HTTP_201_CREATED)
async def register_user(user_data: UserCreate, db: Session = Depends(get_db)):
    user_service = UserService(db)

    if await user_service.get_user_by_email(user_data.email):
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Користувач з таким email вже існує",
        )
    if await user_service.get_user_by_username(user_data.username):
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Користувач з таким іменем вже існує",
        )
    user_data.password = Hash().get_password_hash(user_data.password)
    return await user_service.create_user(user_data)

@router.post("/login", response_model=Token)
async def login_user(
    form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)
):
    user_service = UserService(db)
    user = await user_service.get_user_by_username(form_data.username)
    if not user or not Hash().verify_password(form_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Неправильний логін або пароль",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token = await create_access_token(data={"sub": user.username})
    return {"access_token": access_token, "token_type": "bearer"}

```

`/register` проверяет уникальность email и username, хеширует пароль и создаёт пользователя — возвращает `409 Conflict`, если такой пользователь уже есть. `/login` сверяет пароль и выдаёт `access_token`.

Маршрут «мой профиль» — `/users/me`

Файл `/src/api/users.py`:

```

router = APIRouter(prefix="/users", tags=["users"])

@router.get("/me", response_model=User)
async def me(user: User = Depends(get_current_user)):
    return user

```

Зачем нужен этот маршрут: он позволяет клиенту (например, фронтенду) проверить, авторизован ли пользователь и валиден ли его токен — простым запросом без параметров. В будущем он же может возвращать роли или права пользователя, на основе которых интерфейс решает, что показывать. Также это защита данных: в отличие от маршрута вида `/users/{user_id}`, здесь пользователь может получить только свои собственные данные — те, что зашиты в его токене.

Подключение роутеров

```

from fastapi import FastAPI
from src.api import tags, utils, notes, auth, users

app = FastAPI()

app.include_router(utils.router, prefix="/api")
app.include_router(tags.router, prefix="/api")
app.include_router(notes.router, prefix="/api")
app.include_router(auth.router, prefix="/api")
app.include_router(users.router, prefix="/api")

```

Авторизация в тегах и заметках

Аутентификация готова — теперь нужно, чтобы каждый пользователь видел и менял только свои теги и заметки. Во все методы репозитория добавляется параметр `user: User`, и все запросы к базе данных фильтруются по этому пользователю.

Файл `/src/repository/tags.py` :

```

class TagRepository:
    def __init__(self, session: AsyncSession):
        self.db = session

    async def get_tags(self, skip: int, limit: int, user: User) -> List[Tag]:
        stmt = select(Tag).filter_by(user=user).offset(skip).limit(limit)
        tags = await self.db.execute(stmt)
        return tags.scalars().all()

    async def get_tag_by_id(self, tag_id: int, user: User) -> Tag | None:
        stmt = select(Tag).filter_by(id=tag_id, user=user)
        tag = await self.db.execute(stmt)
        return tag.scalar_one_or_none()

    async def create_tag(self, body: TagModel, user: User) -> Tag:
        tag = Tag(**body.model_dump(exclude_unset=True), user=user)
        self.db.add(tag)
        await self.db.commit()
        await self.db.refresh(tag)
        return tag

    async def update_tag(self, tag_id: int, body: TagModel, user: User) -> Tag | None:
        tag = await self.get_tag_by_id(tag_id, user)
        if tag:
            tag.name = body.name
            await self.db.commit()
            await self.db.refresh(tag)
        return tag

    async def remove_tag(self, tag_id: int, user: User) -> Tag | None:
        tag = await self.get_tag_by_id(tag_id, user)
        if tag:
            await self.db.delete(tag)
            await self.db.commit()
        return tag

```

Каждый запрос теперь ищет данные с условием «и принадлежит этому `user`» — так один пользователь физически не может получить доступ к чужому тегу, даже зная его id. У `NoteRepository` те же изменения: параметр `user` добавлен во все методы, а при создании заметки указывается `user=user`.

В сервисе тегов (`/src/services/tags.py`) дополнительно обрабатывается ошибка нарушения уникальности — когда пользователь пытается создать тег с именем, которое у него уже есть:

```
def _handle_integrity_error(e: IntegrityError):
    if "unique_tag_user" in str(e.orig):
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Ter з такою назвою вже існує.",
        )
    else:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Помилка цілісності даних.",
        )

class TagService:
    async def create_tag(self, body: TagModel, user: User):
        try:
            return await self.repository.create_tag(body, user)
        except IntegrityError as e:
            await self.repository.db.rollback()
            _handle_integrity_error(e)
```

После ошибки `IntegrityError` сессия базы данных остаётся в неконсистентном состоянии — обязательно нужен `rollback()`, чтобы откатить незавершённую транзакцию, прежде чем показать пользователю понятное сообщение об ошибке.

И, наконец, в самих маршрутах добавляется зависимость `Depends(get_current_user)` — так каждый маршрут получает текущего пользователя и работает только в его контексте:

```
@router.get("/", response_model=List[TagResponse])
async def read_tags(
    skip: int = 0,
    limit: int = 100,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    tag_service = TagService(db)
    return await tag_service.get_tags(skip, limit, user)
```

То же самое добавляется во все маршруты тегов и заметок. В результате в Swagger-документации у защищённых маршрутов появляется значок замочка — без валидного токена доступ к ним закрыт.

Проверка работы

Полный цикл проверяется через Postman: регистрация (`POST /api/auth/register`) → вход (`POST /api/auth/login` , форма `x-www-form-urlencoded`) — в ответ приходит `access_token` → создание тега (`POST /api/tags`) и заметки (`POST /api/notes`) с заголовком `Authorization: Bearer {access_token}` . Всё работает так же, как в примерах ранее — только теперь каждая созданная запись автоматически привязана к текущему пользователю.

Переменные окружения

Хранить пароли и секретные ключи прямо в коде — плохая практика: если код попадёт в публичный репозиторий, все секреты утекут вместе с ним. Для этого используют **переменные окружения** — пары «ключ-значение», которые задаются на уровне операционной системы и доступны приложению, но не хранятся в коде.

Базовая работа через модуль os

```
import os

# получить значение переменной окружения
database_url = os.environ.get('DATABASE_URL')

# если переменная не задана — можно указать значение по умолчанию
debug_mode = os.environ.get('DEBUG_MODE', False)
```

Файл .env и библиотека python-dotenv

Для удобства во время разработки переменные окружения хранят в файле `.env` в корне проекта:

```
poetry add python-dotenv
```

```
API_KEY=abcdefghijklmnopqrstuvwxyz
SECRET_KEY=1234567890
DEBUG=True
```

Файл `.env` обязательно нужно добавить в `.gitignore`, чтобы он не попал в репозиторий. Значения в нём пишутся **без кавычек** — кавычки здесь будут ошибкой.

Функция `load_dotenv()` загружает значения из `.env` в `os.environ`, после чего их можно читать обычным способом:

```
import os
from dotenv import load_dotenv

load_dotenv()
api_key = os.environ.get('API_KEY')
secret_key = os.environ.get('SECRET_KEY')
```

Хорошая практика — проверять при старте приложения, что все обязательные переменные заданы, и сразу останавливать запуск с понятной ошибкой, если чего-то не хватает:

```
import os, sys

required_vars = ['DATABASE_URL', 'SECRET_KEY']

for var in required_vars:
    if not os.environ.get(var):
        print(f"Error: {var} is not set.")
        sys.exit(1)
```

Pydantic Settings

В FastAPI-проектах удобнее не читать переменные вручную, а описать их как класс — за это отвечает `BaseSettings` из отдельного пакета `pydantic-settings` (в Pydantic 2.0+ этот класс вынесли из основного пакета):

```
poetry add pydantic-settings
```

Файл `.env` с настройками проекта — обратите внимание на плейсхолдеры вида `${VARIABLE}`, которые подставляют значение одной переменной внутрь другой, чтобы не дублировать данные:

```
POSTGRES_DB=todo_app
POSTGRES_USER=postgres
POSTGRES_PASSWORD=567234
POSTGRES_PORT=5432
POSTGRES_HOST=localhost
```

```
DB_URL=postgresql+asyncpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@${POSTGRES_HOST}:${POSTGRES_PORT}/${POSTGRES_DB}
JWT_SECRET=your_secret_key
JWT_ALGORITHM=HS256
JWT_EXPIRATION_SECONDS=3600
```

Файл `src/conf/config.py` :

```
from pydantic import ConfigDict
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    DB_URL: str
    JWT_SECRET: str
    JWT_ALGORITHM: str = "HS256"
    JWT_EXPIRATION_SECONDS: int = 3600
    model_config = ConfigDict(
        extra="ignore", env_file=".env", env_file_encoding="utf-8", case_sensitive=True
    )

settings = Settings()
```

Класс переименован из `Config` в `Settings` — старое имя было принято в Pydantic v1 для другой цели и могло путать. Поля объявляются так же, как в обычных Pydantic-моделях, с типами и значениями по умолчанию.

ПАРАМЕТР MODEL_CONFIG	ЧТО ДЕЛАЕТ
<code>extra="ignore"</code>	Игнорирует лишние поля, которых нет в классе <code>Settings</code> .
<code>env_file=".env"</code>	Указывает, из какого файла загружать значения.
<code>env_file_encoding="utf-8"</code>	Кодировка файла <code>.env</code> .
<code>case_sensitive=True</code>	Без этого параметра регистр имени переменной не важен (например, <code>db_url</code> тоже работает).

После создания объекта `settings` значения доступны в любом месте кода: `settings.DB_URL` , `settings.JWT_SECRET` и так далее.

Применение settings по проекту

Дальше остаётся заменить все важные значения на обращения к `settings` . Подключение к базе данных,

`src/database/db.py` :

```
from src.conf.config import settings
# ...
sessionmanager = DatabaseSessionManager(settings.DB_URL)
```

Сервис аутентификации, `/src/services/auth.py` :

```
from src.conf.config import settings
# ...
expire = datetime.now(UTC) + timedelta(seconds=settings.JWT_EXPIRATION_SECONDS)
encoded_jwt = jwt.encode(to_encode, settings.JWT_SECRET, algorithm=settings.JWT_ALGORITHM)
# ...
payload = jwt.decode(token, settings.JWT_SECRET, algorithms=[settings.JWT_ALGORITHM])
```

Настройка Alembic, `migrations/env.py` :

```
from src.conf.config import settings
# ...
config.set_main_option("sqlalchemy.url", settings.DB_URL)
```

Переменные окружения в Docker Compose

Файл `docker-compose.yml` тоже может подставлять значения из `.env` через плейсхолдеры `${VARIABLE}` :

```
version: '3'
services:
  postgres:
    image: postgres:12
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - ./postgres-data:/var/lib/postgresql/data
```

При запуске Docker сам подставит значения из `.env` вместо плейсхолдеров. Здесь описан сервис `postgres` на образе `postgres:12`, порт контейнера `5432` проброшен на такой же порт хоста, а данные базы сохраняются в локальную папку `./postgres-data`, чтобы не потерять их при пересоздании контейнера. Запуск — командой `docker-compose up -d`.

Ограничение количества запросов (Rate Limiting)

Зачем ограничивать частоту запросов к маршруту:

- **Защита от DDoS-атак** (Distributed Denial of Service — распределённая атака типа «отказ в обслуживании») — большой поток запросов за короткое время может перегрузить сервер.
- **Защита ресурсов** — маршрут может использовать «дорогие» операции (запросы к базе данных, вычисления), и лишняя нагрузка может их исчерпать.
- **Защита пропускной способности** — если маршрут дергает внешний сервис с собственным лимитом запросов.
- **Ограничение на одного пользователя** — например, чтобы не дать автоматически собирать данные (скрапинг).

Для FastAPI есть несколько готовых библиотек: **fastapi-ratelimiter**, **SlowApi**, **fastapi-limiter**. Все они добавляют небольшую накладную нагрузку на каждый запрос, а слишком агрессивные лимиты могут мешать обычным пользователям — поэтому лимиты нужно подбирать взвешенно.

Для промышленных приложений от DDoS-атак дополнительно защищают на уровне сети — например, сервисом **Cloudflare**, который распределяет трафик через сеть дата-центров и предоставляет защиту от атак прямо перед сервером.

Библиотека slowapi

`slowapi` реализует ограничение скорости без обязательного внешнего хранилища (вроде Redis) — удобно для небольших проектов. В основе — алгоритм **token bucket** («ведро с токенами»): у каждого клиента есть запас «токенов» на запросы, который тратится и восстанавливается со временем. Данные о лимитах хранятся в памяти процесса сервера.

Основные компоненты: `Limiter` — класс, который применяет ограничения; `key_func` — функция, определяющая, по какому признаку считать лимит (например, по IP-адресу); декораторы — задают лимит на конкретном маршруте.

```
poetry add slowapi
```

```
from slowapi import Limiter
from slowapi.errors import RateLimitExceeded
from slowapi.util import get_remote_address
from fastapi import FastAPI, Request
from starlette.responses import JSONResponse

app = FastAPI()
limiter = Limiter(key_func=get_remote_address)

@app.exception_handler(RateLimitExceeded)
async def rate_limit_handler(request: Request, exc: RateLimitExceeded):
    return JSONResponse(
        status_code=429,
        content={"error": "Перевищено ліміт запитів. Спробуйте пізніше."},
    )

@app.get("/my-endpoint")
@limiter.limit("5/minute")
async def my_endpoint(request: Request):
    return {"message": "Це мій маршрут з обмеженням швидкості"}
```

`get_remote_address` определяет пользователя по IP-адресу запроса. Обработчик `rate_limit_handler` срабатывает, когда лимит превышен, и возвращает статус **429 (Too Many Requests)**. Декоратор `@limiter.limit("5/minute")` ограничивает маршрут пятью запросами в минуту.

ФОРМАТ СТРОКИ ЛИМИТА	ЗНАЧЕНИЕ
"5/minute" (или "5 per minute")	5 запросов в минуту
"100/hour"	100 запросов в час
"1000/day"	1000 запросов в день

Поддерживаемые единицы времени: `second` , `minute` , `hour` , `day` . На один маршрут можно навесить сразу несколько ограничений:

```
@app.get("/complex")
@limiter.limit("10/minute")
@limiter.limit("1/second")
async def complex_route(request: Request):
    return {"message": "Складний маршрут"}
```

⚠ `slowapi` хранит счётчики в памяти одного процесса. Если приложение запущено в нескольких процессах или контейнерах (например, несколько воркеров Uvicorn/Gunicorn), у каждого процесса будет свой собственный счётчик — реальный лимит для пользователя окажется выше ожидаемого. Для масштабирования нужно общее хранилище состояния (например, Redis), что потребует дополнительной настройки.

Применение в проекте

Добавим лимит на маршрут `/api/users/me` — файл `/src/api/users.py` :

```
from slowapi import Limiter
from slowapi.util import get_remote_address

router = APIRouter(prefix="/users", tags=["users"])
limiter = Limiter(key_func=get_remote_address)

@router.get(
    "/me", response_model=User, description="No more than 10 requests per minute"
)
@limiter.limit("10/minute")
async def me(user: User = Depends(get_current_user)):
    return user
```

Параметр `description` сразу отражает лимит в Swagger-документации. Осталось зарегистрировать обработчик ошибки превышения лимита в `main.py` :

```
from slowapi.errors import RateLimitExceeded
from starlette.responses import JSONResponse

@app.exception_handler(RateLimitExceeded)
async def rate_limit_handler(request: Request, exc: RateLimitExceeded):
    return JSONResponse(
        status_code=status.HTTP_429_TOO_MANY_REQUESTS,
        content={"error": "Перевищено ліміт запитів. Спробуйте пізніше."},
    )
```

Теперь при более чем 10 запросах в минуту к `/api/users/me` сервер вернёт статус 429.

Cross-Origin Resource Sharing (CORS)

CORS (Cross-Origin Resource Sharing — совместное использование ресурсов между источниками) — механизм, который разрешает или запрещает запросы к ресурсам с других доменов. Он защищает от атак вроде XSS (Cross-Site Scripting — межсайтовый скриптинг) и CSRF (Cross-Site Request Forgery — подделка межсайтового запроса).

По умолчанию браузеры соблюдают **same-origin policy** (политику одного источника): JavaScript-код, загруженный с одного домена, не может обращаться к API на другом домене без явного разрешения. Например, фронтенд работает на `http://localhost:3000`, а API — на `http://localhost:8000`. Чтобы браузер разрешил фронтенду делать запросы к API, сервер должен явно указать, каким источникам («origins») это можно.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost:3000"
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```


Когда браузер отправляет запрос, он добавляет заголовок `Origin` с доменом-источником. Если этот домен есть в списке `origins`, сервер добавляет в ответ заголовок `Access-Control-Allow-Origin` — и браузер разрешает JavaScript получить данные.

ПАРАМЕТР	ЧТО ДЕЛАЕТ
<code>allow_origins</code>	Список доменов, которым разрешён доступ.
<code>allow_credentials</code>	<code>True</code> — разрешает кросс-доменные запросы с передачей учётных данных (cookie, заголовки авторизации).
<code>allow_methods</code>	Разрешённые HTTP-методы; <code>"*"</code> — все методы.
<code>allow_headers</code>	Разрешённые заголовки запроса; <code>"*"</code> — все заголовки.

Можно указать `allow_origins=["*"]`, чтобы разрешить запросы с любого домена — но это ослабляет защиту, поэтому в продакшене обычно указывают конкретный список источников.

Ограничение доступа по IP и User-Agent

Ещё один способ защиты — фильтровать запросы прямо в `middleware` (промежуточном слое обработки), до того как они дойдут до маршрута: по IP-адресу клиента или по строке `User-Agent`.

Чёрный список IP-адресов

```
from ipaddress import ip_address
from fastapi import FastAPI, Request, status
from fastapi.responses import JSONResponse

app = FastAPI()

banned_ips = [
    ip_address("192.168.1.1"),
    ip_address("192.168.1.2"),
    ip_address("127.0.0.1"),
]

@app.middleware("http")
async def ban_ips(request: Request, call_next: Callable):
    ip = ip_address(request.client.host)
    if ip in banned_ips:
        return JSONResponse(
            status_code=status.HTTP_403_FORBIDDEN, content={"detail": "You are banned"}
        )
    response = await call_next(request)
    return response
```

`Middleware` проверяет IP клиента на каждый запрос. Если он в списке `banned_ips` — сразу возвращается ошибка **403 (Forbidden)**, минуя обработчик маршрута. Если нет — запрос идёт дальше через `call_next(request)`.

Белый список IP-адресов

```
ALLOWED_IPS = [ip_address('192.168.1.0'), ip_address('172.16.0.0'), ip_address("127.0.0.1")]

@app.middleware("http")
async def limit_access_by_ip(request: Request, call_next: Callable):
    ip = ip_address(request.client.host)
    if ip not in ALLOWED_IPS:
        return JSONResponse(status_code=status.HTTP_403_FORBIDDEN, content={"detail": "Not allowed IP address"})
    response = await call_next(request)
    return response
```

Здесь логика обратная: доступ разрешён только тем IP, что есть в списке `ALLOWED_IPS`, всем остальным — отказ.

Бан по User-Agent

User-Agent — строка в заголовке запроса, которую браузер (или другой клиент) отправляет серверу; в ней зашита информация о браузере, ОС и т.д. По ней можно, например, заблокировать ботов и скрипты, которые не маскируются под обычный браузер.

```
import re

user_agent_ban_list = [r"Gecko", r"Python-urllib"]

@app.middleware("http")
async def user_agent_ban_middleware(request: Request, call_next: Callable):
    user_agent = request.headers.get("user-agent")
    for ban_pattern in user_agent_ban_list:
        if re.search(ban_pattern, user_agent):
            return JSONResponse(status_code=status.HTTP_403_FORBIDDEN, content={"detail": "You are banned"})
    response = await call_next(request)
    return response
```

Middleware сравнивает заголовок `user-agent` запроса с шаблонами (регулярными выражениями) из чёрного списка. При совпадении — тот же ответ 403, иначе запрос идёт дальше как обычно.

Кеширование веб-приложений (Redis)

Что такое Redis

Redis — очень быстрая база данных, потому что хранит все данные в оперативной памяти (RAM), а не на диске (при этом периодически сохраняет состояние на диск, чтобы уменьшить потери при сбое). Ключевая для кеширования особенность — встроенный **TTL** (Time To Live — время жизни): любой записи можно задать срок, по истечении которого она удаляется автоматически, без ручной инвалидации кеша. Также можно ограничить объём занимаемой памяти — когда лимит исчерпан, Redis сам вытесняет самые старые записи.

Redis поддерживает несколько типов данных: строки, словари (hashes), списки, множества, сортированные множества. Ключом всегда служит строка — если передать что-то другое, Redis сам приведёт это к строке.

Подключение и базовые команды

```
poetry add redis
```

```
import redis

r = redis.Redis(host="localhost", port=6379, password=None)

r.set('foo', 'bar')
value = r.get('foo')
print(value)  # bar
```

Проще всего думать про Redis как про большой словарь, где ключи — всегда строки, а пары ключ-значение могут автоматически удаляться по условию (истекло время, кончилось место).

Кроме `set / get`, у Redis много встроенных команд для работы со списками, счётчиками и другими структурами, а также готовый механизм Publish/Subscribe — когда несколько клиентов подписываются на события, а Redis сам рассылает им сообщения при наступлении события.

LRU-кеширование функций

Локально Redis можно поднять в Docker:

```
docker run --name redis-cache -d -p 6379:6379 redis
```

Чтобы не реализовывать **LRU** (Least Recently Used — вытеснение давно неиспользуемых записей) вручную, есть готовый пакет `redis-lru`:

```
poetry add redis-lru
```

```
import redis
from redis_lru import RedisLRU

client = redis.StrictRedis(host="localhost", port=6379, password=None)
cache = RedisLRU(client)

@cache
def f(x):
    print(f"Function call f({x})")
    return x

print(f"Result f(3): {f(3)}")
print(f"Result f(3): {f(3)}")  # второй раз — из кеша, без вызова функции
```

Декоратор `@cache` сохраняет результат функции в Redis на заданное время (по умолчанию — 15 минут). Время жизни настраивается параметром `default_ttl`, максимальный размер кеша — параметром `max_size` при создании `RedisLRU`.

Принципы кеширования

- Кешировать стоит то, что часто запрашивается, но редко меняется.
- Кеш можно организовать на уровне приложения или на уровне базы данных — в зависимости от задачи.
- Если данные должны обновляться динамически — нужен механизм инвалидации кеша (например, через pub/sub).
- Размер кеша стоит ограничивать и удалять старые записи.
- Главное правило: кешировать только тогда, когда это реально ускоряет приложение.

ПОДХОД	КОГДА УМЕСТЕН
Полный кеш	Данные статичные или почти не меняются — кешируем все ответы, избегая обращений к базе.
Частичный кеш	Кешируются только часто запрашиваемые данные — при высокой нагрузке на базу.
Кеш с тайм-аутом (TTL)	Данные динамические — кешируем на ограниченное время, потом обновляем.

Пример: кеширование ответа в FastAPI

```
import redis, json
from fastapi import FastAPI

app = FastAPI()
r = redis.Redis(host='localhost', port=6379, db=0)

@app.get("/product/{product_id}")
def read_product(product_id: int):
    product = r.get(str(product_id))
    if product is None:
        product = fetch_product_from_db(product_id)
        r.set(str(product_id), json.dumps(product))
        r.expire(str(product_id), 3600)
    return product
return json.loads(product)
```

При первом запросе данных нет в Redis — они берутся из базы данных, сохраняются в Redis командой `set` и получают срок жизни 3600 секунд через `expire`. При повторном запросе того же `product_id` данные приходят сразу из кеша, минуя базу данных.

Выбор между TTL и LRU зависит от характера данных: если данные часто меняются — уместнее LRU (вытеснение по давности обращения), если редко — проще и предсказуемее TTL (фиксированное время жизни).

Управление ролями пользователей

Роли определяют, какие действия и ресурсы доступны конкретному пользователю. Типичная иерархия: **администратор** — полный доступ, **модератор** — управление контентом, **обычный пользователь** — стандартный набор функций, **гость** — базовый доступ без входа в систему.

Напомним разницу: аутентификация устанавливает личность пользователя, авторизация — проверяет права уже установленного пользователя на конкретный ресурс. Контроль по ролям обычно реализуют через middleware или зависимости (dependencies) на бэкенде, а на фронтенде — показом/скрытием элементов интерфейса в зависимости от роли.

Модель и схемы

```

from enum import Enum
from sqlalchemy import Column, Integer, String, Enum as SqlEnum

class UserRole(Enum):
    USER = "user"
    MODERATOR = "moderator"
    ADMIN = "admin"

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True)
    hashed_password = Column(String)
    role = Column(SqlEnum(UserRole), default=UserRole.USER, nullable=False)

```

`UserRole` — перечисление (Enum), которое ограничивает роль строго тремя допустимыми значениями и защищает от опечаток и произвольных строк. В схемах Pydantic (`UserCreate` , `UserModel`) поле `role: UserRole` работает так же — автоматически проверяет, что передано одно из разрешённых значений.

Проверка роли через зависимости

В `auth.py` , поверх обычной проверки токена (`get_current_user`), добавляются зависимости, которые дополнительно проверяют роль:

```

def get_current_moderator_user(current_user: User = Depends(get_current_user)):
    if current_user.role not in [UserRole.MODERATOR, UserRole.ADMIN]:
        raise HTTPException(status_code=403, detail="Недостаточно прав доступа")
    return current_user

def get_current_admin_user(current_user: User = Depends(get_current_user)):
    if current_user.role != UserRole.ADMIN:
        raise HTTPException(status_code=403, detail="Недостаточно прав доступа")
    return current_user

```

Дальше эти зависимости просто подставляются в нужные маршруты вместо обычного `get_current_user` :

```

@app.get("/public")
def read_public():
    return {"message": "Це публічний маршрут, доступний для всіх"}

@app.get("/moderator")
def read_moderator(current_user: User = Depends(get_current_moderator_user)):
    return {"message": f"Вітаємо, {current_user.username}!"}

@app.get("/admin")
def read_admin(current_user: User = Depends(get_current_admin_user)):
    return {"message": f"Вітаємо, {current_user.username}!"}

```

В итоге: `/public` доступен всем без авторизации, `/moderator` — только ролям `moderator` и `admin` , `/admin` — только роли `admin` . Попытка доступа с неподходящей ролью вернёт ошибку **403 (Forbidden)**.

Email-рассылки и отправка писем

Стоит различать два похожих, но разных сценария: **массовая рассылка** (новости, маркетинговые кампании — множеству получателей сразу, обычно через сервисы вроде Mailchimp или SendGrid) и **отправка одного письма** конкретному пользователю (подтверждение регистрации, сброс пароля и т.п.) — обычно прямо из кода бэкенда.

СТАТУС ПИСЬМА	ЧТО ОЗНАЧАЕТ
Delivery (доставлено)	Письмо успешно дошло до получателя.
Bounce (отказ)	Письмо не доставлено — например, из-за неверного адреса.
Spam (спам)	Письмо посчитали нежелательным и отправили в папку «Спам» (или удалили).

Много Bounce и Spam подряд — и сервис рассылки может вовсе заблокировать отправку от вашего имени. Поэтому важны качественный список получателей, осмысленный текст письма и отслеживание результатов рассылки.

Для отправки одного письма из кода на Python есть встроенная библиотека `smtplib`, но удобнее использовать более простые обёртки — например, `yagmail`.

Частые задачи с отправкой писем на бэкенде: подтверждение регистрации со ссылкой активации, сброс пароля, уведомления о новых событиях, статус заказа, а также одноразовые коды подтверждения — **ОТП** (One-Time Password — одноразовый пароль), которые присылают на почту для дополнительной проверки личности пользователя.

Верификация email

1. Пользователь указывает email при регистрации.
2. Система создаёт уникальный токен проверки и отправляет письмо со ссылкой на этот адрес.
3. Пользователь переходит по ссылке из письма.
4. Сервер проверяет, что токен в ссылке действителен и соответствует этому email.
5. Если токен верный — email помечается как подтверждённый, пользователь получает полный доступ.
6. Если токен недействителен — предлагается повторить попытку или запросить письмо заново.

Сброс пароля

1. Пользователь запрашивает сброс пароля, указывая свой email.
2. Сервер создаёт уникальный токен сброса и отправляет письмо со ссылкой, где токен передан параметром.
3. При переходе по ссылке API проверяет токен и показывает форму ввода нового пароля.
4. Пользователь вводит новый пароль и отправляет форму.
5. Сервер сохраняет новый пароль и сообщает об успехе.

Отправка email через FastAPI-Mail

Для отправки писем из кода используется пакет **fastapi-mail**. Письма отправляются через обычный SMTP-сервер почтового ящика (подойдёт любой сервис, который разрешает доступ по протоколу POP3/SMTP для сторонних приложений — у Gmail с этим давно строгие ограничения, поэтому в примерах ниже используется вымышленный ящик `example@meta.ua`).

```
poetry add fastapi-mail
```

```

from pathlib import Path
from fastapi import FastAPI, BackgroundTasks
from fastapi_mail import FastMail, MessageSchema, ConnectionConfig, MessageType
from pydantic import EmailStr, BaseModel

class EmailSchema(BaseModel):
    email: EmailStr

conf = ConnectionConfig(
    MAIL_USERNAME="example@meta.ua",
    MAIL_PASSWORD="secretPassword",
    MAIL_FROM="example@meta.ua",
    MAIL_PORT=465,
    MAIL_SERVER="smtp.meta.ua",
    MAIL_FROM_NAME="Example email",
    MAIL_STARTTLS=False,
    MAIL_SSL_TLS=True,
    USE_CREDENTIALS=True,
    VALIDATE_CERTS=True,
    TEMPLATE_FOLDER=Path(__file__).parent / 'templates',
)

app = FastAPI()

@app.post("/send-email")
async def send_in_background(background_tasks: BackgroundTasks, body: EmailSchema):
    message = MessageSchema(
        subject="Fastapi mail module",
        recipients=[body.email],
        template_body={"fullname": "Billy Jones"},
        subtype=MessageType.html
    )
    fm = FastMail(conf)
    background_tasks.add_task(fm.send_message, message, template_name="example_email.html")
    return {"message": "email has been sent"}

```

`ConnectionConfig` хранит данные подключения к почтовому серверу. `MessageSchema` описывает само письмо: тему (`subject`), получателей и `template_body` — переменные, которые подставляются в HTML-шаблон письма. Шаблон ищется в папке, указанной в `TEMPLATE_FOLDER`.

Отправка идёт через **BackgroundTasks** — фоновую задачу FastAPI. Так долгая операция (отправка письма) не блокирует ответ клиенту: сервер сразу возвращает `{"message": "email has been sent"}`, а письмо уходит уже после этого, в фоне.

Шаблон письма (Jinja2), например `templates/example_email.html`, — обычный HTML, куда через `{{ переменная }}` подставляются значения из `template_body`:

```

<p>Hi {{fullname}},</p>
<p>We invite you to visit our beautiful site</p>
<p><a href="http://localhost:8000">Click here</a></p>

```

Верификация email пользователя (практика)

Реализуем полный цикл подтверждения регистрации: пользователь регистрируется → получает письмо со ссылкой → переходит по ней → email помечается подтверждённым, и только после этого разрешён вход.

Поле `confirmed` и миграция

```
class User(Base):
    # ...
    confirmed = Column(Boolean, default=False)
```

```
alembic revision --autogenerate -m 'add to model User field confirmed'
alembic upgrade head
```

В репозиторий пользователя добавляется метод:

```
async def confirmed_email(self, email: str) -> None:
    user = await self.get_user_by_email(email)
    user.confirmed = True
    await self.db.commit()
```

Сервис отправки письма верификации

Файл `src/services/email.py` :

```
from fastapi_mail import FastMail, MessageSchema, ConnectionConfig, MessageType
from fastapi_mail.errors import ConnectionErrors
from src.services.auth import create_email_token
from src.conf.config import settings

conf = ConnectionConfig(
    MAIL_USERNAME=settings.MAIL_USERNAME,
    MAIL_PASSWORD=settings.MAIL_PASSWORD,
    MAIL_FROM=settings.MAIL_FROM,
    MAIL_PORT=settings.MAIL_PORT,
    MAIL_SERVER=settings.MAIL_SERVER,
    MAIL_FROM_NAME=settings.MAIL_FROM_NAME,
    MAIL_STARTTLS=settings.MAIL_STARTTLS,
    MAIL_SSL_TLS=settings.MAIL_SSL_TLS,
    USE_CREDENTIALS=settings.USE_CREDENTIALS,
    VALIDATE_CERTS=settings.VALIDATE_CERTS,
    TEMPLATE_FOLDER=Path(__file__).parent / "templates",
)

async def send_email(email: EmailStr, username: str, host: str):
    try:
        token_verification = create_email_token({"sub": email})
        message = MessageSchema(
            subject="Confirm your email",
            recipients=[email],
            template_body={"host": host, "username": username, "token": token_verification},
            subtype=MessageType.html,
        )
        fm = FastMail(conf)
        await fm.send_message(message, template_name="verify_email.html")
    except ConnectionErrors as err:
        print(err)
```

В `auth.py` добавляется функция создания токена для письма — со сроком жизни 7 дней (за это время пользователь должен успеть подтвердить почту):


```
def create_email_token(data: dict):
    to_encode = data.copy()
    expire = datetime.now(UTC) + timedelta(days=7)
    to_encode.update({"iat": datetime.now(UTC), "exp": expire})
    return jwt.encode(to_encode, settings.JWT_SECRET, algorithm=settings.JWT_ALGORITHM)
```

Шаблон `src/services/templates/verify_email.html` содержит ссылку с этим токеном:

```
<a href="{{host}}api/auth/confirmed_email/{{token}}">Verification</a>
```

Настройки почты

Класс `Settings` дополняется полями почты, значения — в `.env`:

```
MAIL_USERNAME: EmailStr = "example@meta.ua"
MAIL_PASSWORD: str = "secretPassword"
MAIL_FROM: EmailStr = "example@meta.ua"
MAIL_PORT: int = 465
MAIL_SERVER: str = "smtp.meta.ua"
MAIL_FROM_NAME: str = "Rest API Service"
MAIL_STARTTLS: bool = False
MAIL_SSL_TLS: bool = True
USE_CREDENTIALS: bool = True
VALIDATE_CERTS: bool = True
```

Маршруты

Регистрация теперь ставит письмо в фоновую задачу:

```
@router.post("/register", response_model=User, status_code=status.HTTP_201_CREATED)
async def register_user(
    user_data: UserCreate, background_tasks: BackgroundTasks, request: Request, db: Session = Depends(get_db)
):
    # ... проверки на существование, хеширование пароля ...
    new_user = await user_service.create_user(user_data)
    background_tasks.add_task(send_email, new_user.email, new_user.username, request.base_url)
    return new_user
```

`request.base_url` — это базовый адрес приложения (схема + хост + порт, без пути), например `http://127.0.0.1:8000/` — он подставляется в ссылку письма.

Вход теперь проверяет, что почта подтверждена:

```
if not user.confirmed:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED, detail="Електронна адреса не підтверджена"
    )
```

Маршрут, на который ведёт ссылка из письма:

```
@router.get("/confirmed_email/{token}")
async def confirmed_email(token: str, db: Session = Depends(get_db)):
    email = await get_email_from_token(token)
    user_service = UserService(db)
    user = await user_service.get_user_by_email(email)
    if user is None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Verification error")
    if user.confirmed:
        return {"message": "Ваша електронна пошта вже підтверджена"}
    await user_service.confirmed_email(email)
    return {"message": "Електронну пошту підтверджено"}
```

Функция `get_email_from_token` в `auth.py` просто расшифровывает JWT и достаёт email из поля `sub`, с понятной ошибкой при невалидном токене:

```
async def get_email_from_token(token: str):
    try:
        payload = jwt.decode(token, settings.JWT_SECRET, algorithms=[settings.JWT_ALGORITHM])
        return payload["sub"]
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY, detail="Неправильний токен для перевірки електронної пошти"
        )
```

И маршрут повторной отправки письма — на случай, если письмо потерялось (например, попало в спам):

```
@router.post("/request_email")
async def request_email(
    body: RequestEmail, background_tasks: BackgroundTasks, request: Request, db: Session = Depends(get_db)
):
    user_service = UserService(db)
    user = await user_service.get_user_by_email(body.email)
    if user and user.confirmed:
        return {"message": "Ваша електронна пошта вже підтверджена"}
    if user:
        background_tasks.add_task(send_email, user.email, user.username, request.base_url)
    return {"message": "Перевірте свою електронну пошту для підтвердження"}
```

Ответ «проверьте почту» возвращается **всегда**, даже если такого email нет в базе, а письмо реально отправляется только когда пользователь найден. Так защищаемся от способа проверить, зарегистрирован ли конкретный email в системе.

Хранение файлов в облаке

Файлы пользователей (например, аватары) обычно хранят не на своём сервере, а в облачном хранилище — так проще с производительностью (нагрузка уходит на сторонний сервис), доступностью, местом на диске и стоимостью, плюс появляется встроенное резервное копирование. Популярные варианты: Amazon S3, Google Cloud Storage, Azure Blob Storage. В примере используется **Cloudinary** — с бесплатным лимитом, которого достаточно для учебных задач.

Ключи доступа после регистрации в Cloudinary кладём в `.env`:

```
CLOUDINARY_NAME=cloud_name
CLOUDINARY_API_KEY=12345678
CLOUDINARY_API_SECRET=api_secret
```

```
poetry add cloudinary
```

Класс `Settings` дополняется соответствующими полями (`CLD_NAME` , `CLD_API_KEY` , `CLD_API_SECRET`), как и в предыдущих разделах.

Маршрут загрузки аватара

```
@router.patch("/avatar", response_model=User)
async def update_avatar_user(
    file: UploadFile = File(),
    user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    avatar_url = UploadFileService(
        settings.CLD_NAME, settings.CLD_API_KEY, settings.CLD_API_SECRET
    ).upload_file(file, user.username)

    user_service = UserService(db)
    user = await user_service.update_avatar_url(user.email, avatar_url)
    return user
```

`UploadFile` из `fastapi` принимает загруженный файл. Маршрут защищён — доступен только авторизованному пользователю (`get_current_user`), и метод `PATCH` подчёркивает, что это частичное обновление данных пользователя.

Сервис загрузки в Cloudinary

```
import cloudinary
import cloudinary.uploader

class UploadFileService:
    def __init__(self, cloud_name, api_key, api_secret):
        cloudinary.config(cloud_name=cloud_name, api_key=api_key, api_secret=api_secret, secure=True)

    @staticmethod
    def upload_file(file, username) -> str:
        public_id = f"RestApp/{username}"
        r = cloudinary.uploader.upload(file.file, public_id=public_id, overwrite=True)
        src_url = cloudinary.CloudinaryImage(public_id).build_url(
            width=250, height=250, crop="fill", version=r.get("version")
        )
        return src_url
```

`public_id` строится из имени пользователя — так аватар каждого пользователя лежит по предсказуемому пути и при повторной загрузке (`overwrite=True`) перезаписывает старый. `crop="fill"` обрезает и масштабирует картинку под заданный размер (250×250) без искажения пропорций, а `version` гарантирует, что после перезаписи на странице отобразится именно новая версия картинки.

Оставшиеся методы — простое сохранение полученной ссылки в базу:

```
# UserService
async def update_avatar_url(self, email: str, url: str):
    return await self.repository.update_avatar_url(email, url)

# UserRepository
async def update_avatar_url(self, email: str, url: str) -> User:
    user = await self.get_user_by_email(email)
    user.avatar = url
    await self.db.commit()
    await self.db.refresh(user)
    return user
```